APLIKACJE WEBOWE

Obsługa błędów i logowanie

Błędy w Node i Express · middleware błędów
poziomy logów · co logować, a czego nie

L6

by ***Mateusz Węglewski***

Baza padła o 3 w nocy

`res.status(404).json(…)`

Błędy rozsiane po kodzie

Każdy kontroler sam składa odpowiedź z błędem

Dziś: jedno miejsce

`Error: at /home/…`

Stos dla każdego

Klient widzi ścieżki i nazwy naszych plików

Dziś: bezpieczna odpowiedź

`console.log('błąd')`

Brak śladów

Nie wiadomo kiedy, komu i dlaczego

Dziś: porządne logi

by ***Mateusz Węglewski***

Dwa rodzaje błędów

OPERACYJNE

Przewidzieliśmy je

Złe dane, brak klucza, nie ma takiego ucznia, baza nie odpowiada

`400 · 401 · 404 · 409 · 503`

Obsługujemy i odpowiadamy

PROGRAMISTY

Bug w naszym kodzie

Literówka, undefined.imie, zła nazwa funkcji, zapomniany await

`500 Internal Server Error`

Logujemy i naprawiamy kod



throw, try, catch

```
function srednia(oceny) {
  if (oceny.length === 0)
    throw new Error('Brak ocen');
  return suma(oceny) / oceny.length;
}

try {
  srednia([]);
} catch (err) {
  console.log(err.message); // Brak ocen
  console.log(err.stack);   // gdzie?
}
```

throw przerywa funkcję i leci w górę

catch łapie go wyżej

message: co się stało

stack: plik i linia, skąd przyszedł



Kto złapie błąd?

W handlerze

```
(req, res) => {
  throw new Error()
}
```

Express łapie

Serwer działa dalej, klient dostaje 500

W async/await

```
async (req, res) => {
  await Uczen.wszyscy()
}
```

Express 5 łapie

Express 4 nie łapał: stare tutoriale mają try/catch i next(err)

W callbacku

```
setTimeout(() => {
  throw new Error()
})
```

Nikt nie łapie

Cały proces pada, wszyscy tracą połączenie

LIVE DEMO 1

Psujemy serwer na cztery sposoby

```
// app.js: zakomentuj app.use(obslugaBledow)
$ npm run dev
$ curl localhost:3000/demo/sync
$ curl localhost:3000/demo/literowka
$ curl localhost:3000/demo/async
$ curl localhost:3000/demo/callback
```

Po każdej komendzie: **co dostał klient** i **czy serwer żyje?**



Middleware błędów

requestId → logHttp → json() → trasy → 404 → **błędy**

```
// na samym końcu app.js
app.use(nieZnaleziono);
app.use((err, req, res, next) => {
  res.status(err.status ?? 500)
     .json({ blad: err.message });
});
```

4 argumenty: po tym Express go poznaje

throw albo **next(err)**: skok prosto tutaj



AppError: błąd z kodem HTTP

utils/AppError.js

```
class AppError extends Error {
  constructor(status, message) {
    super(message);
    this.status = status;
  }
}
```

Zwykły Error, który dodatkowo wie, **jaki kod odesłać**

L5: KONTROLER ODPOWIADA

```
if (!uczen) return res.status(404)
  .json({ blad: 'Nie znaleziono' });
```

L6: KONTROLER RZUCA

```
if (!uczen)
  throw new AppError(404, 'Nie znaleziono');
```




Jedno miejsce decyduje

middleware/obslugaBledow.js (skrót)

```
module.exports = (err, req, res, next) => {
  const status = err.status ?? 500;
  const nasz = status >= 500;  // 5xx = bug po naszej stronie

  if (nasz) log.error(err.message, { id: req.id, stos: err.stack });

  res.status(status).json({
    blad: nasz ? 'Wewnętrzny błąd serwera' : err.message,
    id: req.id,
  });
};
```

4xx: klient dostaje komunikat · **5xx**: klient dostaje ogólnik, szczegóły idą do logu



Ten sam błąd 500, dwa odbiorcy

Klient widzi

```
{
  "blad": "Wewnętrzny błąd serwera",
  "id": "9d20577d"
}
```

Że coś poszło nie tak i co podać przy zgłoszeniu

Log zapisuje

```
03:12:07 ERROR 9d20577d
Utracono połączenie z bazą
at zapytanie (uczen.model.js:15)
at lista (uczniowie.controller.js:16)
```

Kiedy, co, gdzie w kodzie i w którym żądaniu

Stos w odpowiedzi to mapa naszego serwera dla atakującego.



LIVE DEMO 2

Wszystkie błędy, jeden format

```
$ curl localhost:3000/api/uczniowie/abc
$ curl -X POST localhost:3000/api/uczniowie \
    -H "Content-Type: application/json" -d '{"imie":'
$ curl -X POST localhost:3000/demo/baza
$ curl localhost:3000/api/uczniowie
Chrome: localhost:3000/nie-ma
```

Przy 500 skopiuj **id** z odpowiedzi i znajdź je w terminalu



Ostatnia linia obrony

server.js

```
process.on('uncaughtException', (err) => {
  log.error('uncaughtException', {
    stos: err.stack,
  });
  process.exit(1);
});

process.on('unhandledRejection', …);
process.on('SIGTERM', () => serwer.close());
```

Stan aplikacji **nieznany**: nie udawaj, że działa

Zapisz log i **zakończ proces**

Podniesie go **menedżer procesów**: pm2, systemd, Docker

SIGTERM: dokończ żądania i wyjdź



Log to jedyny świadek tego, co serwer robił, gdy nikt nie patrzył.

console.log pisze tekst. Logger zapisuje zdarzenia.



by ***Mateusz Węglewski***

Pięć poziomów logów

error	Coś się zepsuło, trzeba zareagować	`Baza nie odpowiada · 500`
warn	Podejrzane, ale aplikacja działa	`401 · 404 · zły JSON`
info	Ważne, normalne zdarzenia	`Serwer wystartował · dodano ucznia`
http	Każde żądanie, jedna linijka	`GET /api/uczniowie 200 25 ms`
debug	Szczegóły dla programisty	`Wartości zmiennych, zapytania`

LOG_LEVEL=warn → zapisuje się warn i wszystko nad nim



Dobry wpis w logu

ZŁY

```
Błąd!
```

DOBRY

```
{"czas":"2026-11-05T03:12:07Z", "poziom":"error",
 "wiadomosc":"Utracono połączenie z bazą", "id":"9d20577d"}
```

KIEDY

Czas w ISO, z datą i strefą

JAK WAŻNE

Poziom

CO

Krótka, stała wiadomość

KONTEKST

Id żądania, status, czas trwania

by ***Mateusz Węglewski***

Dla człowieka i dla maszyny

development: kolorowy tekst, czytasz w terminalu

```
20:49:20 HTTP  GET /api/uczniowie {"id":"1a666cbf","status":200,"ms":25}
20:49:20 WARN  GET /api/uczniowie/99 {"id":"a3cb825a","status":404}
```

production: JSON, jedna linijka = jedno zdarzenie

```
{"czas":"2026-10-29T20:49:36Z","poziom":"warn","wiadomosc":"GET /api/uczniowie/99",
 "id":"ce9fbd5e","status":404,"ms":26}
```

Logi idą na **stdout**. Gdzie trafią dalej, decyduje środowisko: plik, Docker, serwis z logami.



Czego nigdy nie logujemy

- Hasła
- Klucze API i tokeny
- Cookies sesji
- Numery kart i kont
- PESEL, adresy, oceny
- Całe body żądania

```
log.info('Logowanie', { login: 'ala', haslo: 'Tajne!23' })
→ {"wiadomosc":"Logowanie","login":"ala","haslo":"***"}
```

Jedno id, trzy miejsca

middleware/requestId.js

```
const { randomUUID } = require('node:crypto');

module.exports = (req, res, next) => {
  req.id = randomUUID().slice(0, 8);
  res.set('X-Request-Id', req.id);
  next();
};
```

1. Nagłówek **X-Request-Id** w każdej odpowiedzi
2. Pole **id** w odpowiedzi z błędem i na stronie błędu
3. Pole **id** w każdym wpisie logu z tego żądania

„Dostałem błąd 9d20577d” → **grep 9d20577d** → cała historia żądania



LIVE DEMO 3

Logi pod lupą

```
$ curl localhost:3000/demo/poziomy
$ LOG_LEVEL=warn npm run dev
$ NODE_ENV=production npm start --silent | tee app.log
$ grep '"poziom":"error"' app.log
$ grep <id z odpowiedzi> app.log
```

Zmieniamy tylko zmienne środowiskowe. **Kod zostaje ten sam.**



Gotowe narzędzia

`pino`

Logger aplikacji

Szybki, od razu JSON. Najczęstszy wybór w nowych projektach

`≈ nasz utils/log.js`

`winston`

Logger z transportami

Jeden log naraz do konsoli, pliku i zewnętrznej usługi

`≈ nasz utils/log.js`

`morgan`

Log żądań HTTP

Middleware Expressa, jedna linijka na żądanie

`≈ nasz logHttp.js`

Swój logger piszemy, żeby zrozumieć zasadę. **W projekcie bierzecie gotowy.**



Pięć rzeczy do zapamiętania

01 Błędy operacyjne obsługujemy, błędy programisty naprawiamy

02 Kontroler rzuca AppError, odpowiada middleware błędów

03 Klient dostaje kod, ogólnik i id. Stos tylko do logu

04 Wpis w logu: czas, poziom, wiadomość, kontekst

05 Hasła, tokeny i dane osobowe nie trafiają do logów





APLIKACJE WEBOWE

:wq

L6



by ***Mateusz Węglewski***